Models

Contents

Model 1: Decision Tree Model for Air Quality Prediction

1. Introduction

A Decision Tree is a supervised machine learning algorithm used for both classification and regression tasks. In this project, it is applied as a regression model to predict air quality indicators, specifically particulate matter concentrations (PM2.5 and PM10), based on environmental and gas sensor inputs collected from an IoT monitoring system.

The model works by recursively splitting the dataset into smaller subsets based on feature thresholds that minimize prediction error. Each internal node represents a decision rule on a feature (for example, temperature ≤ 28°C), while each leaf node represents a final predicted numerical output.

In the context of air quality monitoring, the decision tree learns relationships between environmental conditions and pollution levels, making it suitable as a baseline predictive model due to its simplicity and interpretability.

2. Dataset Description

The dataset used in this project is obtained from a high-resolution IoT-based air quality monitoring system connected to a MySQL database (`aq_highres`). The dataset consists of continuously collected sensor readings recorded at short time intervals.

The total number of datapoints is:

66,919 records

2.1 Input Features

The model uses the following independent variables:

Temperature (°C)

Humidity (%)

MQ7 ADC values (Carbon Monoxide sensor)

MQ2 ADC values (Gas/Smoke sensor)

MQ8 ADC values (Hydrogen sensor)

2.2 Target Variables

The dependent variables to be predicted are:

PM2.5 concentration (µg/m³)

PM10 concentration (µg/m³)

2.3 Data Preprocessing

Before training the model, the dataset undergoes preprocessing steps including:

Removal of missing or null values

Ensuring numeric consistency across sensor readings

Optional outlier handling using percentile-based clipping

Separation of features (X) and targets (y)

3. Data Splitting Strategy

The dataset is divided into training and testing sets using an 80/20 split ratio.

3.1 Computation of Split

Total dataset size = 66,919 records

Training set (80%):

66,919 × 0.8 = 53,535 records (approximately)

Testing set (20%):

66,919 × 0.2 = 13,384 records (approximately)

3.2 Split Method

A time-aware splitting approach is used, meaning:

Data is not randomly shuffled

Training data consists of earlier time-ordered records

Testing data consists of later unseen records

This approach ensures that the model simulates real-world forecasting conditions where future data is predicted based on past observations.

4. Model Training

The Decision Tree Regressor is trained using the training dataset (53,535 records). The model learns decision rules that minimize prediction error at each split.

4.1 Training Mechanism

At each node, the algorithm:

Selects the best feature (e.g., MQ7, temperature)

Finds the optimal split threshold

Minimizes impurity using Mean Squared Error (MSE)

The process continues recursively until stopping conditions are met.

4.2 Model Parameters

Typical configuration includes:

Maximum depth (e.g., 10–15) to prevent overfitting

Minimum samples per leaf to ensure stability

Split criterion: Mean Squared Error (MSE)

The model essentially learns nonlinear relationships between sensor readings and pollution levels.

5. Model Testing and Prediction

After training, the model is evaluated using the test dataset (13,384 records).

5.1 Prediction Process

The model receives unseen sensor readings

It traverses the decision tree based on learned rules

Outputs predicted PM2.5 and PM10 values

5.2 Comparison

Predicted values are compared with actual measured values from the dataset to determine accuracy.

6. Evaluation Metrics

The model is evaluated using four key performance metrics:

6.1 Mean Absolute Error (MAE)

MAE measures the average magnitude of prediction errors without considering direction.

MAE=1n∑|yactual−ypredicted|MAE = \frac{1}{n} \sum |y_{actual} - y_{predicted}|MAE=n1 ∑|yactual−ypredicted|

Interpretation:

Measures average deviation in absolute terms

Lower MAE indicates better prediction accuracy

6.2 Root Mean Squared Error (RMSE)

RMSE measures the square root of the average squared differences between predicted and actual values.

RMSE=1n∑(yactual−ypredicted)2RMSE = \sqrt{\frac{1}{n} \sum (y_{actual} - y_{predicted})^2}RMSE=n1∑(yactual−ypredicted)2

Interpretation:

Penalizes large errors more heavily than MAE

Useful for detecting large prediction deviations

6.3 Coefficient of Determination (R^2 Score)

R^2 measures how well the model explains variance in the target variable.

R2=1−SSresSStotR^2 = 1 - \frac{SS_{res}}{SS_{tot}}R2=1−SStotSSres

Interpretation:

$R^2 = 1$ → perfect prediction

$R^2 = 0$ → model performs like mean baseline

$R^2 < 0$ → model performs worse than mean prediction

6.4 Mean Absolute Percentage Error (MAPE)

MAPE expresses error as a percentage:

MAPE=1n∑|yactual−ypredictedyactual|MAPE = \frac{1}{n} \sum \left|\frac{y_{actual} - y_{predicted}}{y_{actual}}\right|MAPE=n1∑yactualyactual−ypredicted

Interpretation:

Useful for relative error measurement

Can become unstable when actual values are near zero

7. Interpretation of Results

The performance of the Decision Tree model depends on how well it generalizes unseen data.

From observed results:

MAE is relatively high, indicating noticeable prediction deviation

RMSE is higher than MAE, suggesting presence of larger errors in some predictions

R^2 is low (approximately 0.31), indicating weak explanatory power

Key Observations:

The model captures only basic nonlinear relationships

It struggles with highly dynamic sensor variations

It is sensitive to noise and outliers in the dataset

8. Role in the System

The Decision Tree model serves the following roles in the system:

Baseline regression model for PM2.5 and PM10 prediction

Initial validation tool for data pipeline correctness

Reference model for comparing advanced algorithms (Random Forest, XGBoost, LSTM)

It is also used for quick interpretability during system debugging and early-stage deployment testing.

9. Limitations

The Decision Tree model has several limitations in this application:

High tendency to overfit if not properly constrained

Poor generalization on complex or noisy sensor data

No inherent capability to model time-series dependencies

Unstable predictions when small data variations occur

Due to these limitations, it is not suitable as a final production forecasting model.

10. Conclusion

The Decision Tree regression model provides a simple and interpretable method for predicting air quality parameters from IoT sensor data. It is effective as a baseline model and useful for initial system validation.

However, its performance limitations in handling noise, temporal dependencies, and complex nonlinear relationships make it less suitable for high-accuracy forecasting tasks. It is therefore primarily used as a reference model for evaluating improvements achieved by ensemble methods (Random Forest, XGBoost) and deep learning approaches (LSTM).

Model 2: Random Forest Model for Air Quality Prediction

1. Introduction

Random Forest is an ensemble learning algorithm used for both classification and regression tasks. In this project, it is applied as a regression model to predict air quality indicators, specifically PM2.5 and PM10 concentrations, using environmental and gas sensor data collected from an IoT monitoring system.

Unlike a single decision tree, Random Forest builds multiple decision trees during training and combines their outputs through averaging (for regression). This ensemble approach reduces overfitting and improves prediction stability.

Each tree in the forest is trained on a random subset of the data and uses a random subset of features at each split. This randomness improves generalization and reduces model variance.

2. Dataset Description

The dataset used is sourced from a high-resolution IoT air quality monitoring system stored in a MySQL database (`aq_highres`).

Total Dataset Size:

66,919 records

2.1 Input Features

The model uses the following independent variables:

Temperature (°C)

Humidity (%)

MQ7 ADC (Carbon Monoxide sensor)

MQ2 ADC (Gas/Smoke sensor)

MQ8 ADC (Hydrogen sensor)

2.2 Target Variables

PM2.5 concentration (µg/m³)

PM10 concentration (μg/m³)

2.3 Data Preprocessing

Removal of missing values

Feature-target separation

Optional noise reduction using clipping

No scaling required for tree-based models

3. Data Splitting Strategy

The dataset is divided into:

Training Set (80%)

Testing Set (20%)

3.1 Computation

Total records = 66,919

Training set:

66,919 × 0.8 = 53,535 records (approx.)

Testing set:

66,919 × 0.2 = 13,384 records (approx.)

3.2 Split Method

A time-aware split is used (no shuffling), ensuring:

Training data represents earlier time sequences

Testing data represents future unseen data

Real-world deployment behavior is simulated

4. Model Training

The Random Forest Regressor is trained using the training dataset (53,535 records).

4.1 Training Mechanism

The model operates by:

Building multiple decision trees independently

Each tree is trained on a bootstrap sample of the dataset

At each split, a random subset of features is selected

Final prediction is computed by averaging outputs from all trees

4.2 Model Parameters

Typical configuration includes:

Number of estimators (trees): 100–300

Maximum depth: controlled to avoid overfitting

Feature sampling per split: random subset selection

Criterion: Mean Squared Error (MSE)

5. Model Testing and Prediction

The trained model is evaluated using the test dataset (13,384 records).

5.1 Prediction Process

Each tree generates a prediction independently

Final output is the average of all tree predictions

Predictions are compared with actual PM2.5 and PM10 values

6. Evaluation Metrics

The model is evaluated using:

6.1 Mean Absolute Error (MAE)

Measures average absolute error between predicted and actual values.

Lower MAE indicates higher accuracy.

6.2 Root Mean Squared Error (RMSE)

Measures magnitude of large prediction errors.

More sensitive to outliers than MAE.

6.3 Coefficient of Determination (R^2)

Measures how well the model explains variance in the data.

Values closer to 1 indicate better performance

Negative values indicate poor performance

6.4 Mean Absolute Percentage Error (MAPE)

Expresses error as a percentage of actual values.

Useful for relative comparison of prediction accuracy.

7. Interpretation of Results

Observed performance:

MAE: ~58.04

RMSE: ~100.82

R^2: ~0.47

Key Observations:

Better performance than Decision Tree

Reduced overfitting due to ensemble averaging

More stable predictions across noisy sensor data

Still limited in capturing temporal dependencies

8. Role in the System

Random Forest is used as:

An improved baseline regression model

A more stable predictor compared to Decision Tree

A reference model for ensemble performance comparison

It is suitable for:

Medium-accuracy air quality prediction

Dashboard-level analytics

Real-time inference with moderate computational cost

9. Limitations

Cannot model time-series dependencies

Higher computational cost than single decision tree

Less interpretable due to ensemble complexity

Performance depends on feature quality and tuning

10. Conclusion

Random Forest improves prediction accuracy and stability compared to a single decision tree by using ensemble learning. It reduces variance and improves generalization on noisy IoT sensor data.

However, it still lacks temporal awareness and is not optimal for forecasting future air quality trends. It is best suited as a strong intermediate model between simple regression methods and advanced boosting or deep learning models.

Model 3: XGBOOST Model for Air Quality Prediction

1. Introduction

XGBoost (Extreme Gradient Boosting) is an advanced ensemble learning algorithm based on gradient boosting techniques. It is widely used for structured data prediction tasks due to its high accuracy and regularization capabilities.

In this project, XGBoost is used to predict PM2.5 and PM10 concentrations from IoT sensor data. Unlike Random Forest, which builds trees independently, XGBoost builds trees sequentially, where each new tree corrects errors made by previous trees.

This sequential learning process makes XGBoost more powerful for capturing complex nonlinear relationships in air quality data.

2. Dataset Description

The dataset is sourced from the IoT-based air quality monitoring system stored in MySQL (`aq_highres`).

Total Dataset Size:

66,919 records

2.1 Input Features

Temperature (°C)

Humidity (%)

MQ7 ADC (CO sensor)

MQ2 ADC (Gas/Smoke sensor)

MQ8 ADC (Hydrogen sensor)

2.2 Target Variables

PM2.5 concentration (µg/m³)

PM10 concentration (µg/m³)

2.3 Data Preprocessing

Removal of missing values

Outlier clipping using percentile filtering (optional)

Feature-target separation

Time-ordered structuring for sequence consistency

3. Data Splitting Strategy

The dataset is divided into:

80% Training Set

20% Testing Set

3.1 Computation

Total records = 66,919

Training set:

53,535 records (approx.)

Testing set:

13,384 records (approx.)

3.2 Split Method

A time-aware split is used to ensure:

Training uses historical data

Testing simulates future unseen conditions

No random shuffling is applied

4. Model Training

XGBoost is trained using the training dataset (53,535 records).

4.1 Training Mechanism

The model works by:

Building decision trees sequentially

Each tree minimizes residual errors from previous trees

Gradient descent is used to optimize loss function

Regularization is applied to reduce overfitting

4.2 Model Parameters

Typical parameters include:

Number of estimators: 200–500

Learning rate: 0.01–0.1

Maximum depth: 4–8

Subsampling ratio: 0.7–0.9

Objective function: regression (square error)

5. Model Testing and Prediction

The trained model is evaluated using the test dataset (13,384 records).

5.1 Prediction Process

Each tree contributes incrementally to final prediction

Errors are corrected step-by-step

Final output is aggregated from all boosted trees

6. Evaluation Metrics

The same metrics are used:

6.1 Mean Absolute Error (MAE)

Measures average absolute prediction error.

6.2 Root Mean Squared Error (RMSE)

Penalizes large errors more heavily.

6.3 Coefficient of Determination (R^2)

Measures how well variance is explained.

Negative values indicate poor generalization

6.4 Mean Absolute Percentage Error (MAPE)

Measures relative error percentage.

7. Interpretation of Results

Observed performance:

MAE: ~70.25

RMSE: ~98.19

R^2: ~ -0.37

Key Observations:

Model shows unstable generalization on current dataset

Negative R^2 indicates poor fit on test data

Possible causes:

Insufficient feature engineering

Lack of temporal sequence features

Sensor noise sensitivity

Improper hyperparameter tuning

8. Role in the System

XGBoost is intended for:

High-accuracy predictive modeling

Advanced regression layer in the system

Real-time PM forecasting when properly tuned

However, in its current state, it requires optimization before production deployment.

9. Limitations

Sensitive to hyperparameter tuning

Requires careful feature engineering

Does not inherently model time-series dependencies

Can overfit if improperly configured

Computationally more expensive than simpler models

10. Conclusion

XGBoost is a powerful gradient boosting model capable of capturing complex nonlinear relationships in air quality data. However, its performance depends heavily on proper feature engineering and parameter tuning.

In this project, it serves as a high-capacity regression model, but currently underperforms due to data structure limitations. With improved temporal features and tuning, it has the potential to become the most accurate classical machine learning model in the system.

Model 4: Isolation Forest Model for Air Quality Anomaly Detection

1. Introduction

Isolation Forest is an unsupervised machine learning algorithm used for anomaly detection. Unlike supervised models that learn from labeled outputs, Isolation Forest identifies anomalies by isolating observations that are significantly different from the rest of the dataset.

In this project, Isolation Forest is applied to detect abnormal air quality conditions based on IoT sensor readings. These anomalies may represent dangerous environmental events such as gas leaks, sudden pollution spikes, or sensor malfunctions.

The core idea of the algorithm is that anomalies are easier to isolate than normal data points. The model constructs multiple random trees and isolates observations based on random feature splits.

2. Dataset Description

The dataset used is sourced from a high-resolution IoT air quality monitoring system stored in the MySQL database (`aq_highres`).

Total Dataset Size:

66,919 records

2.1 Input Features

The model uses the following sensor inputs:

Temperature (°C)

Humidity (%)

MQ7 ADC (Carbon Monoxide sensor)

MQ2 ADC (Gas/Smoke sensor)

MQ8 ADC (Hydrogen sensor)

PM2.5 concentration

PM10 concentration

2.2 Data Characteristics

Unlike supervised models, Isolation Forest does not require labeled output variables. It learns patterns of normal behavior from the full dataset.

2.3 Preprocessing

Removal of missing values

Optional normalization (not strictly required)

Feature selection based on sensor relevance

3. Data Splitting Strategy

Isolation Forest does not use a traditional supervised train-test split. Instead:

The model is trained on the entire dataset (66,919 records)

A contamination parameter defines expected anomaly ratio

Evaluation is performed using threshold-based anomaly scoring

4. Model Training

4.1 Training Mechanism

The Isolation Forest algorithm works by:

Randomly selecting a feature

Randomly selecting a split value

Recursively partitioning the dataset

Measuring how quickly a point is isolated

Points that are isolated in fewer splits are considered anomalies.

4.2 Model Parameters

Typical configuration includes:

Number of estimators (trees): 100–300

Contamination rate: estimated anomaly percentage

Maximum samples: subset of dataset per tree

5. Anomaly Detection Process

After training:

Each data point is assigned an anomaly score

Low scores indicate normal behavior

High scores indicate potential anomalies

The system flags abnormal air quality events based on sensor deviations.

6. Evaluation Metrics

Since this is an unsupervised model, evaluation is not based on traditional regression metrics. Instead, evaluation uses:

6.1 Anomaly Score Distribution

Measures how data points are distributed between normal and abnormal regions

6.2 Threshold-Based Validation

Domain thresholds are applied (e.g., MQ7 high values)

Model outputs are compared against known environmental rules

6.3 Detection Consistency

Stability of anomaly detection across time-series data

7. Interpretation of Results

The model successfully identifies extreme deviations in sensor readings

High MQ sensor spikes are consistently flagged as anomalies

Performance depends heavily on contamination parameter selection

8. Role in the System

Isolation Forest is used for:

Real-time anomaly detection

Early warning system for hazardous air conditions

Triggering alerts and actuator responses (fans, alarms, notifications)

It does not perform prediction but acts as a safety and monitoring layer.

9. Limitations

Does not predict future values

Sensitive to parameter tuning (contamination rate)

Can misclassify rare but valid environmental conditions

No temporal awareness of sensor data

10. Conclusion

Isolation Forest provides an efficient and scalable method for detecting abnormal air quality conditions in IoT sensor data. It plays a critical role in identifying unexpected environmental events and supporting real-time alert systems. However, it is not suitable for forecasting and must be combined with predictive models for a complete system.

Model 5: LSTM Model for Air Quality Forecasting

1. Introduction

Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) designed for sequential and time-series data analysis. It is specifically capable of learning long-term dependencies in temporal datasets.

In this project, LSTM is used for forecasting future air quality levels (PM2.5 and PM10) based on historical IoT sensor readings. Unlike traditional machine learning models, LSTM considers the sequential nature of the data, making it highly suitable for real-time environmental prediction.

The model is designed to learn patterns such as daily pollution cycles, sensor drift behavior, and gradual environmental changes.

2. Dataset Description

The dataset originates from an IoT-based air quality monitoring system stored in MySQL (`aq_highres`).

Total Dataset Size:

66,919 records

2.1 Input Features

The model uses sequential time-series inputs:

Temperature (°C)

Humidity (%)

MQ7 ADC (CO sensor)

MQ2 ADC (Gas/Smoke sensor)

MQ8 ADC (Hydrogen sensor)

2.2 Target Variables

PM2.5 concentration

PM10 concentration

2.3 Data Transformation

Before training, the dataset is converted into sequences:

Sliding window approach (e.g., 10–60 time steps)

Each input sample contains past sensor readings

Output is the next future value of PM2.5/PM10

3. Data Splitting Strategy

The dataset is split using an 80/20 time-series approach:

3.1 Computation

Total records = 66,919

Training set:

53,535 records (approx.)

Testing set:

13,384 records (approx.)

3.2 Split Method

No shuffling is applied

Earlier sequences are used for training

Later sequences are used for testing

This preserves temporal order for realistic forecasting

4. Model Training

4.1 Training Mechanism

The LSTM model learns using:

Input sequences of sensor data

Memory cells that retain long-term dependencies

Gated mechanisms (input, forget, output gates)

The model learns how pollution evolves over time rather than isolated snapshots.

4.2 Model Architecture

Typical architecture includes:

One or more LSTM layers

Dense output layer for regression

Loss function: Mean Squared Error (MSE)

Optimizer: Adam

5. Model Testing and Prediction

5.1 Prediction Process

Input: past sensor sequences

Output: predicted future PM2.5 and PM10 values

Predictions are compared against actual future readings

5.2 Forecasting Capability

The model can predict:

Short-term air quality trends

Future pollution spikes

Gradual environmental changes

6. Evaluation Metrics

The model is evaluated using:

6.1 Mean Absolute Error (MAE)

Measures average prediction deviation.

6.2 Root Mean Squared Error (RMSE)

Measures magnitude of large forecasting errors.

6.3 Coefficient of Determination (R^2)

Measures how well the model explains variance in time-series data.

6.4 Training vs Validation Loss

Used to evaluate:

Overfitting behavior

Learning stability

Model convergence

7. Interpretation of Results

LSTM is expected to outperform classical models in forecasting tasks

Captures temporal dependencies that tree-based models cannot

Performance depends heavily on sequence length and data quality

8. Role in the System

LSTM serves as:

Primary forecasting model

Early warning system for pollution trends

Input to intelligent alert and actuator systems

Advanced analytics layer in the dashboard

9. Limitations

Requires high computational resources

Sensitive to hyperparameter tuning

Requires properly structured sequential data

Training time is longer compared to classical models

10. Conclusion

LSTM is the most suitable model in this project for predicting future air quality conditions due to its ability to learn temporal patterns in sensor data. It enables forecasting of PM2.5 and PM10 levels, making it critical for early warning systems and proactive environmental control in the IoT air quality monitoring system.